The Metropolitan Post

VOL. 1, NO. 097

2026-03-30

FEATURES

Using Heartbleed as a starting point

Antirez

The strong reactions about the recent OpenSSL bug are understandable: it is not fun when suddenly all the internet needs to be patched. Moreover for me personally how trivial the bug is, is disturbing. I don't want to point the finger to the OpenSSL developers, but you just usually think at those class of issues as a bit more subtle, in the case of a software like OpenSSL. Usually you fail to do sanity checks *correctly*, as opposed to this bug where there is a total *lack* of bound checks in the memcpy() call.

However sometimes in the morning I read the code I wrote the night before and I'm deeply embarrassed. Programmers sometimes fail, I for sure do often, so my guess is that what is needed is a different process, and not a different OpenSSL team.

There is who proposes a different language safer than C, and who proposes that the specification is broken because it is too complex. Probably there is some truth in both arguments, however it is unlikely that we move to a different specification or system language soon, so the real question is, what we can do now to improve system software security?

1) Throw money at it.

Making system code safer is simple if there are investments. If different companies hire security experts to do code auditings in the OpenSSL code base, what happens is that the probability of discovering a bug like heartbleed is greater.

I've seen very complex bugs that are triggered by a set of non-trivial conditions being discovered by serious code auditing efforts. A memcpy() without bound checks is something that if you analyze the code security-wise, will stand out in the first read. And guess how heartbleed was discovered? Via security auditings performed at Google.

Probably the time to consider open source something that mostly we take from is over. Many companies should follow the example of Google and other companies, using workforce for OSS software development and security.

2) Static and dynamic checks.

Static code analysis is, as a side effect, a semi-automated way to do code auditings.

In critical system code like OpenSSL even to do some source code annotation or use a set of rules to make static analysis more effective is definitely acceptable.

Static tools today are not a total solution, but the output of a static analysis if carefully inspected by an expert programmer can provide some value.

Another great help comes from dynamic checks like Valgrind. Every system software written in C should be tested using Valgrind automatically at every new commit.

3) Abstract C with libraries.

C is low level and has no built in safety in the language. However something good about C is that it is a language that allows to build layers on top of its rawness.

A sane dynamic string library prevents a lot of buffer overflow issues, and today almost every decent project is using one. However there is more you can do about it. For example for security critical code where memory can contain things like private keys, you can augment your dynamic string library with memory copy primitives that only copy from one buffer to the other performing implicit sanity checks.

Moreover if a buffer contains critical data, you can set logical permissions so that trying to copy from this area aborts the program. There are other less-portable ways using memory management to protect important memory pages in an even more effective ways, however an higher C-level protection can be much simpler in the real-world because of portability / predictability concerns.

In general many things can be explored to avoid using C without protections, creating a library that abstracts on top of it to make programming safer.

4) Randomized tests.

Unit tests are unlikely to trigger edge cases and failed sanity checks.

There is a class of tests that is known since decades that is, in my opinion, not used enough: fuzzy testing.

The OpenSSL bug was definitely discoverable by sending different kind of OpenSSL packets with different randomized parameters, in conjunction with dynamic analysis tools like Valgrind.

In my experience having a great deal of randomized tests together with an environment where the same tests are ran again and again with the program running over Valgrind, can discover a number of real-world bugs that gets otherwise unnoticed. There are many models to explore, usually you want something that injects totally random data, and intermediate models where valid packets are corrupted in different random ways.

A typical example of this technique is the old DNS compression infinite-loop bug. Trow a few random packets to a naive implementation and you'll find it in a matter of minutes.

5) Change of mentality about security vs performance.

It is interesting that OpenSSL is doing its own allocation caching stuff because in some systems malloc/free is slow. This is a sign that still performances, even in security crit-

First Token Cutoff LLM sampling

Antirez

From a theoretical standpoint, the best reply provided by an LLM is obtained by always picking the token associated with the highest probability. This approach makes the LLM output deterministic, which is not a good property for a number of applications. For this reason, in order to balance LLMs creativity while preserving adherence to the context, different sampling algorithms have been proposed in recent years.

Today one of the most used ones, more or less the default, is called top-p: it is a form of nucleus sampling where top-scoring tokens are collected up to a total probability sum of "p", then random weighted sampling is performed.

In this blog post I'll examine why I believe nucleus sampling may not be the best approach, and will show a simple and understandable alternative in order to avoid the issues of nucleus sampling. The algorithm is yet a work in progress, but by publishing it now I hope to stimulate some discussion / hacking.

There is some gold in the logits

Despite the fact that LLM logits are one of the few completely understandable parts of the LLM inner working, I generally see very little interest in studying their features, investigating more advanced sampling methods, detecting and signaling users uncertainty and likely hallucination. Visualizing the probabilities distribution for successive tokens is a simple and practical exercise in order to gain some insights:

! !

In the image we can see the top 32 candidate tokens colored by probability (white = 0, blue = 1), the selected token and the rank of the selected token (highest probability = 0, the previous one = 1, and so forth).

In the above example, the Mistral base model knows the birth and death dates of Umberto Eco, so it confidently signals the most likely token with most of the total probability. Other times the model is more perplexed because either there are multiple ways to express the continuation of the text, or because it is not certain about certain facts. Asking the date of a name which birthday was not learned during training produces a different distribution of token probabilities.

However in general what we want to avoid is to select suboptimal tokens, putting the LLM generation outside the path of minimum perplexity and hallucination. Nucleus sampling fails at doing this because accumulating tokens up to "p", depending on the distribution may include tokens that are extremely weaker than the first choice. Consider, for instance, the "writer" token in the image above. Umberto Eco was a writer, indeed. The token associated with writer, while it is not scoring with a very high value, it is still a lot more above the second choice. Yet a p of 0.3 may accumulate the second token with a low value like 0.01, and yield it with some single-digit probability, with the risk of putting the generation in the wrong path.

Instead, in the case of the following token, "who", there are multiple choices with a relatively similar score, that could be used for alternative generations.

Making use of the avalanche effect

A key observation here is that to select too weak tokens for the sake of variability is not a good deal: even if we only exploit generations moments where there are a few good candidates in order to diversify the output, the avalanche effect will help us: the input context will change, thus the output of the LLM will be perturbed, with the effect of making it more likely to produce some alternative version of the text.

First Token Cutoff (FTC) algorithm

DISCLAIMER: The algorithm described here hasn't undergone any scientific scrutiny. I experimented a couple of days with different sampling algorithms having "bound worst token" properties, and this one looks like the best balance between applicability, results and understandability.

The algorithm described here can be informally stated as follows:

- When the LLM is strongly biased towards a given candidate, select it.
- When there are multiple viable candidates, produce alternatives.
- The selection of the worst possible token should be bounded to a given amount.

Past work, like Tail Free Sampling, also noted that a selection should be made across a small set of high-quality tokens emitted by the LLM. However in TFS such set is identified by performing the derivative to select a cluster corresponding to the tokens that don't see a steep curve after which the token quality decreases strongly.

In the algorithm proposed here, instead, we want the selection to follow a more bounded and understandable cut-off relative to the highest scoring token T0, attributing to T0 a special meaning compared to all the other tokens: the level of certainty the LLM has during the emission of such token (a proxy of perplexity, basically). Thus the algorithm refuses every token that is worse than a given percentage if compared to T0.

The cut off percentage, that can have a value from 0 to 1, is called "co".

An example and viable "co" could be 0.5.

Lesson Two: Life Doesn’t Need to Be Exhausting

Scott Young · Scott Young (learning)

Next week, I’m opening registration for my new course Everyday Energy . In the last essay , I discussed why we’re living through a crisis of human energy. Today, I’d like to share some thoughts on how to fix that, with insights drawn from the first month of the three-month curriculum.

Energy starts with biology. All human physical and cognitive performance is constrained by the fundamental reality that each of us exists as a biological system. When we incur debts—in sleep, stamina or stress—that remain unpaid for too long, the result is breakdown of that system.

Data show that, biologically-speaking, there’s a lot we could be doing better to enjoy greater vitality.

We’re in denial about our fitness . According to self-reports, over half of Americans meet the recommended amounts of weekly exercise. But, when that figure was measured objectively using wearable devices, only one-in-ten actually met the recommendation .

And the recommendations aren’t even optimal! Indeed, while most of us would benefit from exercising more (and a few extreme individuals would benefit from exercising less), the recommended amount of exercise is still below the amount where health benefits plateau .

We sleep poorly. Roughly a third of us fail to get enough sleep . One in ten of us have insomnia . Even worse, our sleep consistency is a disaster. Nearly 85% of people exhibit some degree of “social jet lag” with diverging weekend and weekday sleep rhythms that have a negative impact on health and energy.

The culprit is biological. The clock cells in our brains run slightly longer than a normal twenty-four hour day. Strong light cues provided by the sun constantly adjust that clock backwards. But, in our environment of bright indoor lights, insufficient daytime sunshine, and non-stop screens, we don’t get the resetting signal. Instead, we live in a state of perpetual jet lag.

Our food is fast and low-quality . While our ancestors had to worry about caloric deficits, for most of us the problem is caloric excess. Ultra-processed diets lead to energy that spikes and crashes, and subtle micronutrient deficiencies can leave us tired all the time without us knowing why.

We live in an era of anxiety . Stress in bursts energizes us, but sustained stress leaves us exhausted . Non-stop psychological threats, increasing social isolation, and fewer outlets for coping have resulted in an explosion of mental health diagnoses, and the energy-sapping effects they entail.

Turning Back the Cycle

Worse, these continual shocks to our system can form a vicious cycle. We get overwhelmed at work, so we cut back on exercise. This degrades our body’s ability to turn down the stress response. The increase in stress causes us to eat more junk food and keeps us up at night. The cycle turns, and we end up even more exhausted than we were before.

The self-reinforcing cycle of fatigue can be debilitating. We lack the very energy we need to do the things we need to gain greater energy!

The solution is to take small steps to turn the vicious cycle of energy sapping into a virtuous cycle of energy replenishment. Go to sleep ten minutes earlier. Take a brief daily walk. Learn some relaxation exercises. Switch from a late-afternoon energy drink to a healthy snack.

Small changes, implemented consistently, can build up to a great deal more energy . While the overall trends undermining the biological sources of our energy are dire, they are not destiny. Each of us has the power to live in the way our body and brain were designed—even in our modern world.

In my upcoming course, Everyday Energy , we will spend the first month making concrete changes in our daily practices to build more energy. You’ll learn the science of caffeine, the biology of stress, the neuroscience of light, and even researched-based answers to questions like whether standing desks work or which dietary changes science backs as actually boosting energy.

Energy starts with biology, but it doesn’t stop there. In the next lesson, I’ll cover some ideas drawn from the second month of the course, flow, where we dive into the psychology of work and rest, and how we can build sustainable rhythms to do the work we need to get done.

The post Lesson Two: Life Doesn’t Need to Be Exhausting appeared first on Scott H Young .

Redis new data structure: the HyperLogLog

Antirez

Generally speaking, I love randomized algorithms, but there is one I love particularly since even after you understand how it works, it still remains magical from a programmer point of view. It accomplishes something that is almost illogical given how little it asks for in terms of time or space. This algorithm is called HyperLogLog, and today it is introduced as a new data structure for Redis.

Counting unique things

===

Usually counting unique things, for example the number of unique IPs that connected today to your web site, or the number of unique searches that your users performed, requires to remember all the unique elements encountered so far, in order to match the next element with the set of already seen elements, and increment a counter only if the new element was never seen before.

This requires an amount of memory proportional to the cardinality (number of items) in the set we are counting, which is, often absolutely prohibitive.

There is a class of algorithms that use randomization in order to provide an approximation of the number of unique elements in a set using just a constant, and small, amount of memory. The best of such algorithms currently known is called HyperLogLog, and is due to Philippe Flajolet.

HyperLogLog is remarkable as it provides a very good approximation of the cardinality of a set even using a very small amount of memory. In the Redis implementation it only uses 12kbytes per key to count with a standard error of 0.81%, and there is no limit to the number of items you can count, unless you approach 2^64 items (which seems quite unlikely).

The algorithm is documented in the original paper [1], and its practical implementation and variants were covered in depth by a 2013 paper from Google [2].

[1] http://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf

[2] http://static.googleusercontent.com/media/research.google.com/en//pubs/archive/40671.pdf

How it works?

===

There are plenty of wonderful resources to learn more about HyperLogLog, such as [3].

[3] http://blog.aggregateknowledge.com/2012/10/25/sketch-of-the-day-hyperloglog-cornerstone-of-a-big-data-infrastructure/

Here I'll cover only the basic idea using a very clever example found at [3]. Imagine you tell me you spent your day flipping a coin, counting how many times you encountered a non interrupted run of heads. If you tell me that the maximum run was of 3 heads, I can imagine that you did not really flipped the coin a lot of times. If instead your longest run was 13, you probably spent a lot of time flipping the coin.

However if you get lucky and the first time you get 10 heads, an event that is unlikely but possible, and then stop flipping your coin, I'll provide you a very wrong approximation of the time you spent flipping the coin. So I may ask you to repeat the experiment, but this time using 10 coins, and 10 different piece of papers, one per coin, where you record the longest run of heads. This time since I can observe more data, my estimation will be better.

Long story short this is what HyperLogLog does: it hashes every new element you observe. Part of the hash is used to index a register (the coin+paper pair, in our previous example. Basically we are splitting the original set into m subsets). The other part of the hash is used to count the longest run of leading zeroes in the hash (our run of heads). The probability of a run of N+1 zeroes is half the probability of a run of length N, so observing the value of the different registers, that are set to the maximum run of zeroes observed so far for a given subset, HyperLogLog is able to provide a very good approximated cardinality.

The Redis implementation

===

The standard error of HyperLogLog is 1.04/sqrt(m), where "m" is the number of registers used.

Redis uses 16384 registers, so the standard error is 0.81%.

Since the hash function used in the Redis implementation has a 64 bit output, and we use 14 bits of the hash output in order to address our 16k registers, we are left with 50 bits, so the longest run of zeroes we can encounter will fit a 6 bit register. This is why a Redis HyperLogLog value only uses 12k bytes for 16k registers.

Because of the use of a 64 bit output function, which is one of the modifications of the algorithm that Google presented in [2], there are no practical limits to the cardinality of the sets we can count. Moreover it is worth to note that the error for very small cardinalities tend to be very small. The following graph shows a run of the algorithm against two different large sets. The cardinality of the set is shown in the x axis, while the relative error (in percentage) in the y axis.

img://antirez.com/misc/hll_1.png

The red and green lines are two different runs with two totally unrelated sets. It shows how the error is consistent as the cardinality increases. However for much smaller cardinalities, you can enjoy a much smaller error:

The open source paradox

Antirez

A new idea is insinuating in social networks and programming communities. It's the proportionality between the money people give you for coding something, and the level of demand for quality they can claim to have about your work.

As somebody said, the best code is written when you are supposed to do something else [1]. Like a writer will do her best when writing that novel that, maybe, nobody will pay a single cent for, and not when doing copywriting work for a well known company, programmers are likely to spend more energies in their open source side projects than during office hours, while writing another piece of a project they feel stupid, boring, pointless. And, if the company is big enough, chances are it will be cancelled in six months anyway or retired one year after the big launch.

Open source is different, it's an artifact, it's a transposition in code of what you really want to do, of what you feel software should be, or just of all your fun and joy, or even anger you are feeling while coding. And you want it to rock, to be perfect, and you can't sleep at night if there is a fucking heisenbug. So if a user of your software is addressing you because some part of your code sucks, and is willing to work with you to do something about it, and is very demanding, don't think they are abusing you because they are not paying you. It's not about money. You can ignore bugs if you want, and ignore their complains, you can do that since you don't have a contract to do otherwise, but they are helping you, they care about the same thing you care: your software quality, grandiosity, perfection.

The real right you have, and often don't exploit, is that you are the only one that can decide about the design of your software. So you are entitled to refuse a pull request, or a proposal to follow good practices, because you feel that what somebody is contributing does not fit in the big picture of what you are designing and building.

But if you recognize that somebody is talking you about something that is, really, a defect in your software, don't do the error of reducing the interaction to a vile matter of money. You are doing work for free, they are risking their asses deploying what you wrote, you both care about quality.

EDIT: If you write OSS and you are upset about user demands, have you ever thought that maybe, at this point, your work is more similar to office work for some reason?

EDIT 2: A HN user asked the reasons for such title. The paradox is that the OSS writer cares and is often willing to fix code she writes for free, more than the other paid work she does.

[1] "The best programs are the ones written when the programmer is supposed to be working on something else." - Melinda Varian. https://twitter.com/CodeWisdom/status/1309470447667421189 Comments

The real right you have, and often don't exploit, is that you are the only one that can decide about the design of your software.

Antirez

How Horses Shaped the Mughal Empire

Livia Gershon · JSTOR Daily

The Mughal Empire became a dominant power in South Asia from the sixteenth through the eighteenth century thanks largely to its cavalry. And, as political scientist Deepti Kumari writes, the importance of horses influenced many aspects of how the empire developed .

Kumari writes that horses were both a source of military strength and a focus of pageantry for Mughal aristocratic leaders. Imperial officers had two ranks: zat , determined by their pay rate, and sawar , reflecting the number of horsemen under their command.

Transporting horses from Kandahar through Multan to the Mughal central city of Agra could mean traveling through a desert, leaving a caravan with no source of water for three to four days.

However, South Asia was considered a poor area for horse breeding due to the heat and monsoons. Instead, the empire largely imported the horses it needed from Arabian, Persian, and Central Asian sources. In exchange, India exported commodities like cotton cloth , indigo , silk, sugar, slaves, spices , and medicines, including opium .

Kumari writes that horses from Central Asia mainly arrived in India over land routes, brought by merchants from nomadic cultures who traveled in caravans. This could be tough going. Transporting horses from Kandahar through Multan to the Mughal central city of Agra , for example, could mean traveling through a desert, leaving a caravan with no source of water for three to four days. Travelers more often took an alternate route across the Khyber Pass in the mountains of what is now Pakistan, even though this added an extra ten days to the journey.

More to Explore

The Supernatural Horses That Fascinated Chinese Emperors

December 2, 2024

In the second century BCE, Han Dynasty Emperor Wu so desired a herd of “blood-sweating” horses from Central Asia that he was willing to wage war over them.

To import horses across land routes, the Mughals built roads and set up travelers’ inns known as caravanserais . They also established diplomatic relationships with countries along the way and recruited locals to be part of the imperial machinery. When Emperor Akbar took power in 1556, he moved the empire’s capital to Lahore in an effort to exercise greater control over trade routes from Central Asia, particularly in the area around the key trading city of Kabul.

“By providing the security, Akbar improved and accelerated the bilateral caravan traffic coming in and going out of India via Kabul and Qandahar route and restored the confidence of the merchants,” Kumari writes. “This ensured a regular supply of the horses and other commodities for the Mughal army and Indian population.”

Meanwhile, horses from Arabia, Persia, and northeastern Africa mostly reached India by sea—routes that had been used since the early medieval period. The Mughals particularly valued these horses for their strength and stamina, as well as their colors and other aesthetic features.

Δ

This made sea ports along the western coast of the subcontinent particularly important. The trading ports were a crucial part of the imperial administration, with their own governors who regulated commercial activity and reported back to the Mughal court.

And within the empire, the Mughals encouraged horse traders by providing them with tax relief and protection while traveling. Emperor Akbar ordered the construction of special quarters for the most trusted horse-dealers to protect them from robbers or bad weather.

“The Mughals encouraged horse-trading and invested in horse-breeding because it was crucial for state building,” Kumari writes.

The post How Horses Shaped the Mughal Empire appeared first on JSTOR Daily .

Module Federation Pipeline—Part 1

Naman Saini · Housing.com Engineering

Module Federation Pipeline — Part 1

What is Module Federation?

Module Federation is a feature in modern JavaScript module bundlers like Webpack that allows separate applications or micro-frontends to share code and resources seamlessly. It enables the development of modular and independent components that can be distributed across different applications, allowing for greater flexibility, code reuse, and improved performance.

With Module Federation, applications can dynamically load and consume modules from other applications, eliminating the need to bundle everything into a single monolithic application. This approach promotes a more scalable and maintainable architecture, as changes or updates to individual modules can be deployed independently without affecting the entire system.

Current architecture of Housing

In our current architecture, we utilize module federation exclusively on the client side. This means that any page or route we want to fetch from our server should not be a federated module, but rather a part of the host app that hosts that specific route.

For instance, when opening the homepage of housing.com at https://housing.com/ , it is considered a part of our demand application. In this scenario, the demand application is responsible for fetching the container files of all other applications within our micro-frontend setup. This process allows us to obtain different modules from sources other than the demand application.

You can observe this in action in the screenshot below.

Other containers being fetched on homepage apart from demand

The container file mentioned above is generated during the build step of Webpack. However, in order for this functionality to work, we need to add a specific plugin and configure it accordingly.

To achieve this, we include a plugin called ModuleFederationPlugin in our Webpack configuration. This plugin plays a crucial role in enabling module federation. By adding the plugin and specifying its configuration, we can ensure that our applications can share code and resources seamlessly.

Implementation steps:

In order to enable module federation, we need to add the ModuleFederationPlugin to the Webpack configuration of each application in our setup. It's important to note that this configuration is specific to the client build of our application and does not apply to the server side. The reason for this is that the ModuleFederationPlugin is designed to support client-side federation only.

This plugin requires some configuration, and there are various options available based on our needs. Here are a few important ones. Please refer to the sample configuration code at the end.

i. 'name': Specifies a unique name for the container (host) project. This name will serve as the key to add this specific container to the window object.

ii. 'filename': Acts as the filename for the container. When we provide a URL to fetch this container from another app, it will end with ...${filename}.js.

iii. 'exposes': An object that specifies the modules exposed by the current project, making them accessible for remote projects. For example, in the code snippet below, this container exposes a module located at the path "./routes/demandRoutes" with the name "exposedRoutes".

iv: 'remotes': Defines the remote modules consumed by the current project. It allows us to specify the remote entry points and the exposed modules. This is used for static remotes when we want to fetch remote modules statically. For instance, in the code snippet below, whenever Webpack encounters 'supply' in an import path, it fetches the supplyContainer from the URL mentioned in the 'remotes' key and extracts the respective module from this container.

v: 'shared': Specifies the shared modules and their versions (using the 'requiredVersion' key). Shared modules are loaded only once, even if multiple projects depend on them. There is also a 'singleton' key that, when set to true, indicates that only a single instance of this shared module will be created among different apps. Otherwise, multiple instances would be created for each federated app, but the code of this module would only be loaded once. For example, in the code snippet below, the 'react' version 18.2.0 is set to be a singleton.

```
const { ModuleFederationPlugin } = require('webpack').container; module.exports = { plugins: [ new ModuleFederationPlugin({ name: 'demand', filename: `demndContainer.m.m.js`, exposes: { exposedRoutes: "./routes/demandRoutes" }, remotes: { supply: 'supply@https://supply-app.com/supplyContainer.js', inventory: 'inventory@https://inventory-app.com/inventoryContainer.js', } shared: { react: { singleton: true, requiredVersion: '18.2.0' }, 'react-redux': { singleton: true, requiredVersion: '8.0.2' }, }), ], };
```

3. If we have specified the 'remotes' configuration in our ModuleFederationPlugin, there is no need for any additional steps. Webpack will automatically fetch the container and its respective module when it encounters an import statement that matches our remotes.

However, if we haven't specified [illegible] it indicates that we want to dynamically fetch a container. Initialize the exposed

GNU and the AI reimplementations

Antirez

Those who cannot remember the past are condemned to repeat it. A sentence that I never really liked, and what is happening with AI, about software projects reimplementations, shows all the limits of such an idea. Many people are protesting the fairness of rewriting existing projects using AI. But, a good portion of such people, during the 90s, were already in the field: they followed the final part (started in the '80s) of the deeds of Richard Stallman, when he and his followers were reimplementing the UNIX userspace for the GNU project. The same people that now are against AI rewrites, back then, cheered for the GNU project actions (rightly, from my point of view – I cheered too).

Stallman is not just a programming genius, he is also the kind of person that has a broad vision across disciplines, and among other things he was well versed in the copyright nuances. He asked the other programmers to reimplement the UNIX userspace in a specific way. A way that would make each tool unique, recognizable, compared to the original copy. Either faster, or more feature rich, or scriptable; qualities that would serve two different goals: to make GNU Hurd better and, at the same time, to provide a protective layer against litigations. If somebody would claim that the GNU implementations were not limited to copying ideas and behaviours (which is legal), but "protected expressions" (that is, the source code verbatim), the added features and the deliberate push towards certain design directions would provide a counter argument that judges could understand.

He also asked to always reimplement the behavior itself, avoiding watching the actual implementation, using specifications and the real world mechanic of the tool, as tested manually by executing it. Still, it is fair to guess that many of the people working at the GNU project likely were exposed or had access to the UNIX source code.

When Linus reimplemented UNIX, writing the Linux kernel, the situation was somewhat more complicated, with an additional layer of indirection. He was exposed to UNIX just as a user, but, apparently, had no access to the source code of UNIX. On the other hand, he was massively exposed to the Minix source code (an implementation of UNIX, but using a microkernel), and to the book describing such implementation as well. But, in turn, when Tanenbaum wrote Minix, he did so after being massively exposed to the UNIX source code. So, SCO (during the IBM litigation) had a hard time trying to claim that Linux contained any protected expressions. Yet, when Linus used Minix as an inspiration, not only was he very familiar with something (Minix) implemented with knowledge of the UNIX code, but (more interestingly) the license of Minix was restrictive, it became open source only in 2000. Still, even in such a setup, Tanenbaum protested about the architecture (in the famous exchange), not about copyright infringement. So, we could reasonably assume Tanenbaum considered rewrites fair, even if Linus was exposed to Minix (and having himself followed a similar process when writing Minix).

What the copyright law really says

To put all this in the right context, let's zoom in on the copyright's actual perimeters: the law says you must not copy "protected expressions". In the case of the software, a protected expression is the code as it is, with the same structure, variables, functions, exact mechanics of how specific things are done, unless they are known algorithms (standard quicksort or a binary search can be implemented in a very similar way and they will not be a violation). The problem is when the business logic of the programs matches perfectly, almost line by line, the original implementation. Otherwise, the copy is lawful and must not obey the original license, as long as it is pretty clear that the code is doing something similar but with code that is not cut & pasted or mechanically translated to some other language, or aesthetically modified just to look a bit different (look: this is exactly the kind of bad-faith maneuver a court will try to identify). I have the feeling that every competent programmer reading this post perfectly knows what a *reimplementation* is and how it looks. There will be inevitable similarities, but the code will be clearly not copied. If this is the legal setup, why do people care about clean room implementations? Well, the reality is: it is just an optimization in case of litigation, it makes it simpler to win in court, but being exposed to the original source code of some program, if the exposition is only used to gain knowledge about the ideas and behavior, is fine. Besides, we are all happy to have Linux today, and the GNU user space, together with many other open source projects that followed a similar path. I believe rules must be applied both when we agree with their ends, and when we don't.

AI enters the scene

So, reimplementations were always possible. What changes, now, is the fact they are brutally faster and cheaper to accomplish. In the past, you had to hire developers, or to be enthusiastic and passionate enough to create a reimplementation yourself, because of business aspirations or because you wanted to share it with the world at large.

Now, you can start a coding agent and proceed in two ways: turn the implementation into a specification, and then in a new session ask the agent to reimplement it, possibly forcing specific qualities, like: make it faster, or make the implementation incredibly easy to follow and understand (that's a good trick to end with an implementation very far from others, given the fact that a lot of code seems to be designed for the opposite goal), or more modular, or resolve this fundamental limitation of the original implementation: all hints that will make it much simpler to significantly diverge from the original design. LLMs, when used in this way, don't produce copies of what they saw in the past, but yet at the end you can use an agent to verify carefully if there is any violation, and if any, replace the occurrences with novel code.

Discovering patterns with React hooks

Maciek Pekala · Pony Foo

One of the things I enjoy the most about coding is discovering patterns. Being aware and mindful of the patterns emerging in your codebase can make it easier to keep that codebase consistent, readable and easy to navigate. Most patterns will remain unspoken, and some, that prove notably elegant, are named and promoted as best practices. Others might even be candidates for a basis of a new abstraction, although I recommend restraint as it's worth remembering that abstractions are expensive .

If you're not familiar with the topic or React hooks, there is plenty of introductory as well as in-depth materials out there. I can also shamelessly recommend my talk on hooks from a recent React Copenhagen meetup. What follows assume at least some understanding of what hooks are and how to use them.

Pattern extinction level event

Discovering new patterns is relatively rare when working with established technologies, but every now and then a new idea comes up that stirs things up within some ecosystem. Enter React hooks. Hooks enable a drastically different way to build stateful components in React, which means that they also invalidate 1 a lot of the established patterns. However, being an exceptionally well thought-through and composition-friendly abstraction, hooks enabled new, elegant patterns to emerge.

Over the last few months, I've been building apps with hooks and I've been having a lot of fun discovering these patterns. I'd like to share one, that I've grown especially fond of. For the lack of a better name, and for sole the purpose of this blog post, let's call it the Choco pattern (from Context, HOok, COpomonent). The pattern comes useful when you need a UI widget which is top-level, centralised and unique across the application and needs to expose some functionality to the rest of the components in the app. It might sound vague at the moment, so let's try to go through solving a concrete problem where I noticed the Choco pattern manifest itself.

Exhibit A: Feature toggles

Let's try to build a simple feature toggle functionality for a React app. There should be UI where the user can toggle features on and off and we want to expose these settings to any component interested. We can identify parts we will need:

a context provider exposing the feature toggles anywhere in the component tree,

a component rendering the UI for toggling features,

a place to store the feature toggle state and the associated logic.

Our first attempt could look like this:

This code is fairly simple and works so we could stop here. However, there is a big opportunity for refactoring, by moving all the code related to the "feature toggle concern" to a separate module, instead of mixing it with the rest of our app. Let's do just that.

The biggest change here is extracting the state and its related logic into a custom hook. We can now clearly see the Choco pattern here, with the Context, HOok, and COpomonent exported from the new module.

I made a choice here to keep the initialFeatures dictionary with the application code instead of moving it to the newly created module. One could argue either way, but by keeping that dictionary out of the feature toggle module and dependency-injecting it, we make that module completely generic and a candidate for possible extraction into a library.

Again, we could easily end the refactoring here. In fact, that's the level of decoupling I'd recommend for in this scenario (more on that later). We can, however, take it one step further and introduce an abstraction to hide some of the implementation details and reduce the API surface of the module. Here is one way to do this:

The biggest difference to notice is that the fact we're using the native React context is now an implementation detail, not exposed to the rest of the app. By adding our own abstraction on top of it we also gained a more fine-grain control over what properties of the context are exposed. In this case, we could expose only the feature toggle dictionary, while keeping the function to toggle them private.

Another thing to notice is that by wrapping the context provider in a component, we had an opportunity to render the FeatureToggler in that component and take that responsibility away from the module's consumer. This might be a viable option if we decide to e.g. render the toggler in a portal to make it overlay the page after the user presses some specific key combination. However, in this example I chose not to - this way is much more flexible as the consumer can choose where and how to render the toggler.

> *In fact, that's the level of decoupling I'd recommend for in this scenario (more on that later).*
>
> Maciek Pekala

What's interesting is that we kept the same type of exports from the feature toggle module (Context, HOok, and COpomonent), but they are consumed in a different way. The hook is now taking care of exposing the features to the components. The context (or more precisely the wrapped context provider) now also encapsulates the state and related logic.

Did we improve this code in comparison to the previous

Misfire

Alex Russell (infrequently noted)

The W3C Technical Architecture Group 1 is out with a blog post and an updated Finding regarding Google's recent announcement that it will not be imminently removing third-party cookies.

The current TAG members are competent technologists who have a long history of nuanced advice that looks past the shouting to get at the technical bedrock of complex situations. The TAG also plays a uniquely helpful role in boiling down the guidance it issues into actionable principles that developers can easily follow.

All of which makes these pronouncements seem like weak tea. To grok why, we need to walk through the threat model, look at the technology options, and try to understand the limits of technical interventions.

Finding A Way Forward

But before that, I should stipulate my personal position on third-party cookies: they aren't great!

They should be removed from browsers when replacements are good and ready, and Google's climbdown isn't helpful. That said, we have seen nothing of the hinted-at alternatives, so the jury's out on what the impact will be in practice. 2

So why am I dissapointed in the TAG, given that my position is essentially what they wrote? Because it failed to acknowledge the limited and contingent upside of removing third-party cookies, or the thorny issues we're left with after they're gone.

So, what do third-party cookies do? And how do they relate to the privacy theat model?

Like a lot of web technology, third-party cookies have both positive and negative uses. Owing to a historcal lack of platform-level identity APIs, they form the backbone of nearly every large Single Sign-On (SSO) system. Thankfully, replacements have been developed and are being iterated on .

Unfortunately, some browsers have unilaterally removed them without developing such replacements, disrupting sign-in flows across the web, harming users and pushing businesses toward native mobile apps. That's bad, as native apps face no limits on communicating with third parties and are generally worse for tracking. They're not even subject to pro-user interventions like browser extensions. The TAG should have called out this aspect of the current debate in its Finding, encouraging vendors to adopt APIs that will make the transition smoother.

The creepy uses of third-party cookies relate to advertising. Third-party cookies provide ad networks and data brokers the ability to silently reidentify users as they browse the web. Some build "shadow profiles" , and most target ads based on sites users visit. This targeting is at the core of the debate around third-party cookies.

Adtech companies like to claim targeting based on these dossiers allows them to put ads in front of users most likely to buy, reducing wasted ad spending. The industry even has a shorthand: "right people, right time, right place."

Despite the bold claims and a consensus that "targeting works," there's reason to believe pervasive surveillence doesn't deliver, and even when it does, isn't more effective.

Assuming the social utility of targeted ads is low — likely much lower than adtech firms claim — shouldn't we support the TAG's finding? Sadly, no. The TAG missed a critical opportunity to call for legislative fixes to the technically unfixable problems it failed to enumerate.

Privacy isn't just about collection, it's about correlation across time. Adtech can and will migrate to the server-side, meaning publishers will become active participants in tracking, funneling data back to ad networks directly from their own logs. Targeting pipelines will still work, with the largest adtech vendors consolidating market share in the process.

This is why "give us your email address for 30% discount" popups and account signup forms are suddenly everywhere. Email addresses are stable, long-lived reidentifiers. Overt mechanisms like this are already replacing third-party cookies. Make no mistake: post-removal, tracking will continue for as long as reidentification has perceived positive economic value. The only way to change that equation is legislation; anything else is a band-aid.

Pulling tracking out of the shadows is good, but a limited and contingent good. Users have a terrible time recognising and mitigating risk on the multi-month time-scales where privacy invasions play out. There's virtually no way to control or predict where collected data will end up in most jurisdictions, and long-term collection gets cheaper by the day.

Once correlates are established, or "consent" is given to process data in ways that facilitate unmasking, re-identification becomes trivial. It only takes giving a phone number to one delivery company, or an email address to one e-commerce site to suddenly light up a shadow profile, linking a vast amount of previously un-attributed browsing to a user. Clearing caches can reset things for a little while, but any tracking vendor that can observe a large proportion of browsing will eventually be able to join things back up.

Removal of third-party cookies can temporarily disrupt this reidentification while collection funnels are rebuilt to use "first party" data, but that's not going to improve the situation over the long haul. The problem isn't just what's being collected now , it's the ocean of dormant data that was previously slurped up. 3 The only way to avoid pervasive collection and reidentification over the long term is to

Put Names and Dates On Documents

Alex Russell (infrequently noted)

Anyone who has worked closely with me, or followed on social media [▯ , ▯], will have seen a post or comment to the effect of:

Names and dates on docs. Every time. Don't forget.

This is most often tacked onto design documents lacking inline attribution, and is phrased provocatively to make it sticky.

Why do I care enough about this to be prescriptive — bordering on pushy — when colleagues accuse me of being Socratic to a fault about everything else? Because not only is unattributed writing a reliable time-waster, the careers of authors hang in the balance.

"These blocks detract from readability."

"There's already an edit log."

"Our team is small.

"Specifying an author elides the contributions of collaborators."

Having to work to find the best person to discuss a topic with is annoying, but in large organisations, the probability of authors having work appropriated without credit goes up when they fail to claim ownership of their writing. It should go without saying that this is toxic, and that functional engineering cultures look harshly on it. But to ward off bad behaviour, it helps to model what's healthy.

The best reminder to cite work is for authors to name themselves. Doing this increases their stature, unsubtly encourages others to link and cite, and leaves a trail of evidence for authors to reference when building a case for promotion.

The importance of evidence to support claims of design work in technical fields cannot be overstated. Having served on hiring and promotion committees for many years, I can unequivocally say that this collateral is often pivotal. The difference between "[x] promote" and "[x] do not promote" often comes down to the names on documents. Reviewers will pay attention to both the authors list and the propensity of design doc authors to cite others. 1

In response to unsubtle nudges, several recurring arguments are offered against explicit authorship notices.

"These blocks detract from readability."

This is fair, but does not outweigh the needs of future readers who may be working to trace a chain of events or ideas. Nor does it outweigh the needs of authors for credit regarding their work at a future date.

"There's already an edit log."

This crutch fails in any number of ways:

PDFs and printed copies do not include authorship data that is not in body text.

Some systems (e.g. Google Docs) do not make the history available to non-editors.

Documents may be copied or re-published in ways that disconnect the content from the original revision tracking system; e.g., in a systems transition as part of an acquisition.

Moreover, design is a complex and collaborative process. Ideas and concepts captured in documents are not always the contribution of the person writing down the conclusions of a whiteboarding session. A clear, consistent way to give credit helps everyone feel included and encourages future collaboration.

"Our team is small.

This is usually offered as a claim of superfluousness. If everyone knows everything and everyone working in a system, why does attribution matter?

Perhaps a team is small now, but will it always be? I am not in a position to tell, and my interlocutors also lack crystal balls. Given the downside risks, attribution is a pittance of an insurance premium.

"Specifying an author elides the contributions of collaborators."

This is easily countered with invitations in comments and drafts for contributors to add themselves to the authorship section. Generous and collaborative folks — the sorts of people we want to promote — reliably add their collaborators to documents proactively and set the expectation that others will do the same. Over time, practice becomes habit, which crystallises into culture.

A final concern I hear is that these blocks require a great deal of upkeep. Long-form revision logs might be onerous, but the minimum viable attribution style only needs three elements: names, emails, and dates. These should be provided on the very first page, ideally just below the title.

The primary date always refers first drafting, even if that is before publication. If deemed necessary, authors can optionally add a "Last Updated" field below the primary date, but this is optional. Documents authored in a single sitting by a lone author should avoid extra visual noise.

Revision logs are generally unnecessary, and my personal view is that they distract from content; if they're a requirement (e.g., in a heavily regulated industry), record them in an Appendix, but do not remove minimum viable attributions.

Deals: Take Up to $199 Off Apple's M1 MacBook Pro and MacBook Air at Best Buy and Amazon

Mitchel Broussard · MacRumors

Best Buy and Amazon are both offering great deals on numerous models of the MacBook Pro and MacBook Air today, including a handful of record low deals on the latest Apple notebooks.

Note: MacRumors is an affiliate partner with some of these vendors. When you click a link and make a purchase, we may receive a small payment, which helps us keep the site running.

13-Inch M1 MacBook Air

Starting with the 2020 M1 13-inch MacBook Air, you'll find the 256GB notebook for $899.00 at Amazon and $899.99 at Best Buy, down from $999.00. Amazon stock has dwindled and will "ship soon," but Best Buy has plenty of stock in all three colors.

Additionally, both retailers have the 512GB M1 MacBook Air at $1,099.99 , down from $1,249.00. While this is a record low price on the 512GB notebook, the 256GB model's discount is a second-best price.

M1 MacBook Air, 256GB - $899.99, down from $999.00 at Best Buy / Amazon

M1 MacBook Air, 512GB - $1,099.99, down from $1,249.00 at Best Buy / Amazon

13-Inch M1 MacBook Pro

For the newest MacBook Pro models, Best Buy and Amazon have lowest ever prices on the new 2020 M1 13-inch MacBook Pro. You can get the 256GB notebook for $1,099.99 , down from $1,299.00, in both Silver and Space Gray.

For more storage, the 512GB M1 MacBook Pro is on sale for $1,299.99 , down from $1,499.00. This model is also available in both Silver and Space Gray at this price.

M1 MacBook Pro, 256GB - $1,099.99, down from $1,299.00 at Best Buy / Amazon

M1 MacBook Pro, 512GB - $1,299.99, down from $1,499.00 at Best Buy / Amazon

You can find even more discounts on other MacBooks by visiting our Best Deals guide for MacBook Pro and MacBook Air. In this guide we track the steepest discounts for the newest MacBook models every week, so be sure to bookmark it and check back often if you're shopping for a new Apple notebook.

Related Roundup: Apple Deals

This article, " Deals: Take Up to $199 Off Apple's M1 MacBook Pro and MacBook Air at Best Buy and Amazon " first appeared on MacRumors.com

Discuss this article in our forums

Deals: Get Apple's 512GB 27-Inch iMac for Lowest Price of $1,699.99 ($299 Off)

Mitchel Broussard · MacRumors

Amazon this week is still hosting a record low deal on Apple's 512GB 27-inch 5K iMac with 6-core CPU. You can get this 2020 model for $1,699.99 , down from $1,999.00, after an automatic coupon worth $199.01 is applied at checkout .

Note: MacRumors is an affiliate partner with Amazon. When you click a link and make a purchase, we may receive a small payment, which helps us keep the site running.

This sale is particularly notable because it knocks down the 512GB 27-inch iMac to the same price level as the 256GB model. It's also the best price we've ever tracked across all of the major Apple resellers online. The iMac is ready to ship today with Amazon's typical free shipping for all Prime members.

$299 OFF

27-inch iMac (512GB SSD) for $1,699.99

You can keep track of ongoing sales on Apple's iMac line by visiting our Best iMac Deals guide. There, we keep track of the best iMac offers from Amazon, Adorama, B&H Photo, and other retailers, so be sure to check back often if you're shopping for an iMac for the first time, or thinking of upgrading.

Related Roundups: Apple Deals , iMac

Buyer's Guide: iMac (Don't Buy)

Related Forum: iMac

This article, " Deals: Get Apple's 512GB 27-Inch iMac for Lowest Price of $1,699.99 ($299 Off) " first appeared on MacRumors.com

Discuss this article in our forums

00 at Best Buy / Amazon M1 MacBook Air, 512GB - $1,099.

Mitchel Broussard

Deals: Get the M1 MacBook Air for Up to $150 Off, Starting at $899 for 256GB

Mitchel Broussard · MacRumors

Amazon, B&H Photo, and Adorama today are discounting the M1 MacBook Air to match previous record low prices for both 256GB and 512GB storage options. To start, you can get the 256GB model for $899.00 today on Adorama, down from an original price of $999.00.

Note: MacRumors is an affiliate partner with Amazon. When you click a link and make a purchase, we may receive a small payment, which helps us keep the site running.

Only Silver and Space Gray are available at this price on Adorama. You can also find this sale on Amazon , with a $50 automatic coupon applied at checkout on the Gold and Silver colors. Gold is available to ship in one to two business days, and Silver will be in stock soon, according to Amazon.

$100 OFF

M1 MacBook Air (256GB) for $899.00

Likewise, the 512GB version of the M1 MacBook Air is seeing a notable discount to $1,099.00 , down from $1,249.00. This is a record low price for the notebook, and it's available in all colors. For Silver and Space Gray, you'll find an automatic coupon that will be applied to your order at the checkout screen. The same sale can be found at B&H Photo on the Gold model.

$150 OFF

M1 MacBook Air (512GB) for $1,099.00

You can find even more discounts on other MacBooks by visiting our Best Deals guide for MacBook Pro and MacBook Air. In this guide we track the steepest discounts for the newest MacBook models every week, so be sure to bookmark it and check back often if you're shopping for a new Apple notebook.

Related Roundups: Apple Deals , MacBook Air

Buyer's Guide: MacBook Air (Buy Now)

Related Forum: MacBook Air

This article, " Deals: Get the M1 MacBook Air for Up to $150 Off, Starting at $899 for 256GB " first appeared on MacRumors.com

Discuss this article in our forums

Deals: Apple's 512GB M1 MacBook Pro Hits $1,299.99 on Amazon ($199 Off, Lowest Price)

Mitchel Broussard · MacRumors

At Amazon today you can get Apple's 13-inch M1 MacBook Pro (512GB) for $1,299.99 , down from $1,499.00 on Amazon. You'll see this price at the checkout screen after an automatic coupon worth $50 is applied.

Note: MacRumors is an affiliate partner with Amazon. When you click a link and make a purchase, we may receive a small payment, which helps us keep the site running.

This sale is available in both Silver and Space Gray color options, and it's in stock and ready to ship. At a total of $199 off the original price, this is a match of the previous low price on this model of the MacBook Pro.

$199 OFF

M1 MacBook Pro (512GB) for $1,299.99

If anyone's on the hunt for the entry level version of the M1 MacBook Pro, Amazon does have the 256GB model at its typical sale price of $1,149.99 , down from $1,299.00. There's no checkout coupon for this one, and it's also being discounted in both colors.

You can find even more discounts on other MacBooks by visiting our Best Deals guide for MacBook Pro and MacBook Air. In this guide we track the steepest discounts for the newest MacBook models every week, so be sure to bookmark it and check back often if you're shopping for a new Apple notebook.

Related Roundup: Apple Deals

Related Forum: MacBook Pro

This article, " Deals: Apple's 512GB M1 MacBook Pro Hits $1,299.99 on Amazon ($199 Off, Lowest Price) " first appeared on MacRumors.com

Discuss this article in our forums

class="fancybutton"> $199 OFF M1 MacBook Pro (512GB) for $1,299.

Mitchel Broussard

ANALYSIS

IN BRIEF

Scientific American, Scientific American: Engineers explain how a collision between an Air Canada plane and a fire truck at one of New York's busiest airports turned deadly

Spencer Ackerman, Wired Science: Months into a supposed ceasefire in Gaza, doctors still have to smuggle in basic medical supplies—and treat new casualties of war.

The Metropolitan Post

Vol. 1, No. 097 · 2026-03-30

Curated and composed automatically.